House Price Prediction API

California, USA - Model serving, load testing and monitoring report

Author: Muhammad Faizan Farooq | GitHub: github.com/faizanjutt473/House_Prediction_Price-California-USA



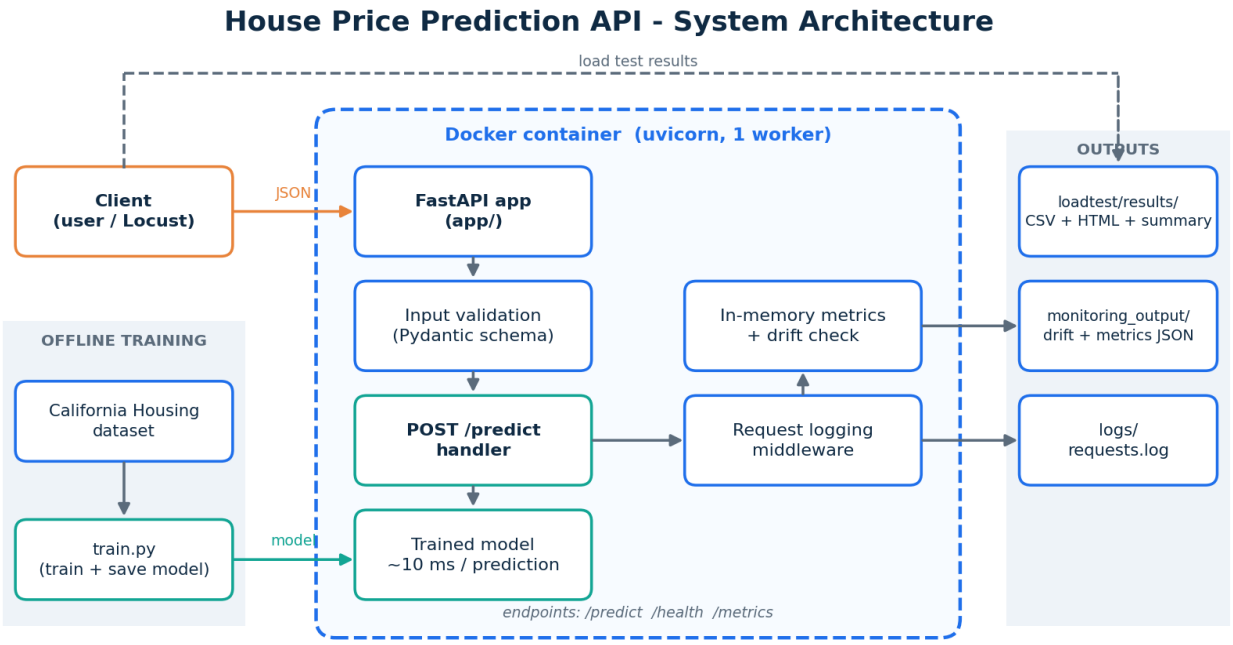
1. Overview

This project trains a house price prediction model on the California housing data and serves it through a FastAPI application packaged in a Docker container. Beyond the model itself, the project focuses on how the service behaves in practice: every request is logged, simple metrics are kept in memory, input drift is checked against normal and shifted traffic, and the API was load tested with Locust at three traffic levels.

Key results

- **0 failures** in 5,048 requests across three load tests.
- **Model inference about 10 ms** per prediction (9.8 ms measured directly on the laptop, 10.1 ms inside the container).
- **Capacity about 33-40 requests/second** for one uvicorn worker on a Windows laptop.
- **Single client, one request at a time:** median round trip 62 ms, p95 106 ms.

2. Architecture



How a request flows

- The client (a user or Locust) sends a JSON request to the FastAPI app running inside the Docker container.
- The input is validated against a schema, then the **/predict** handler calls the trained model.
- A logging middleware records each request to `logs/requests.log`; metrics and drift statistics are kept in memory.
- `train.py` runs offline: it trains the model on the California housing data and saves it for the API to load.
- Results of monitoring and load tests are written to `monitoring_output/` and `loadtest/results/`.

Project contents

Path	Purpose
app/	FastAPI application code
train.py	Model training script
Dockerfile	Container image for the API
scripts/	Helper scripts (sequential_check.py, summarize_loadtest.py)
loadtest/results/	Locust CSV and HTML reports, summary.md
monitoring_output/	Drift (normal vs shifted traffic) and metrics JSON files
logs/requests.log	Request log from the container
README.md	Setup and results documentation

3. Load Test Results

Environment: Windows laptop, Docker Desktop with 4 CPUs available, API in one container with 1 uvicorn worker. Locust ran on the same machine and competes with the API for CPU. Latencies are measured by Locust (client side).

Test	Users	Requests	Failures	Throughput (req/s)	Median (ms)	p95 (ms)	p99 (ms)	Max (ms)
Baseline	2	806	0	13.8	81	160	220	318
Light load	10	2,323	0	39.8	190	320	450	1,706
Saturation	50	1,919	0	32.6	1,500	1,900	2,100	2,738

Latency columns are for the /predict endpoint.

Single client check (no concurrency)

Measurement	Median	p95
Inference time reported by the API, one request at a time	12.4 ms	35.9 ms
Model run directly (no API)	9.8 ms	n/a
Full request round trip	62.2 ms	106 ms

What the results show

- **No failures** at any load level.
- **Throughput stops growing between 10 and 50 users** (about 40 vs 33 req/s). Extra users only wait in a queue, which is why the median rises from 190 ms to 1,500 ms.
- **The model itself is fast.** Docker adds no meaningful overhead to the prediction (9.8 ms on the laptop vs 10.1 ms in the container).
- **Inference time grows under concurrency** (median about 33 ms with 2 users, about 104 ms with 10-50 users) because simultaneous requests and Locust share the same 4 CPUs.
- Locust numbers (client side) are higher than the server log numbers (for example 81 ms vs 57.7 ms at baseline) because Locust also includes the Docker network path.

4. Limitations and Next Steps

- Locust and the API share one machine, so the absolute numbers are pessimistic.
- About 50 ms between model time and round-trip time in the single-client test was not profiled.
- The 1.7 s maximum in the 10-user test is a single outlier request; its cause was not investigated.
- To handle more traffic: run several containers behind a load balancer and move the in-memory metrics to Prometheus or Redis.